Assignment 2
Computational Methods for Network Science
NETS/PHYS 7052, Fall 2026

Haley Broadus https://github.com/Mangbunn/nets7052_fa26

1. *Question 1.*

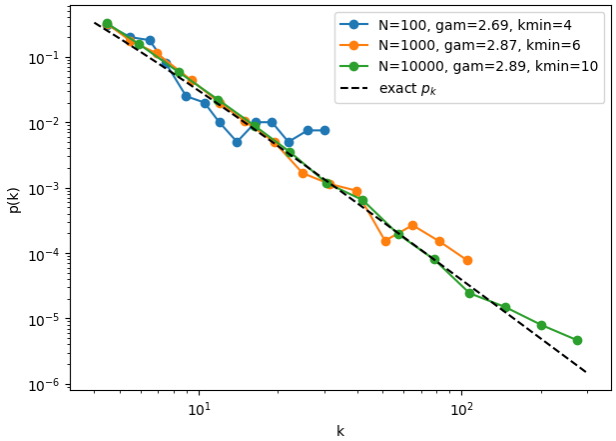


FIG. 1. The relation between k and $p(k)$ in a log-log plot. The dashed line represent the exact distribution of p(k) for our model. each of the colors corresponds to a snapshot of the model at $N = 100, 1000, 10000$ nodes respectively. Using a maximum likelihood estimate of γ above the kmin cutoff.

(a)

(b) Yes, as N increase we see that γ approaches three from bellow.

TABLE I. Power-law fit (`powerlaw`, discrete) to the degree distribution of a single BA network ($m = 4$) at three sizes.

N	$\hat{\gamma}$	$k_{\min}$
10^2	2.69	4
10^3	2.87	6
10^4	2.89	10

The exact degree distribution is

$$p_k = \frac{2m(m+1)}{k(k+1)(k+2)} = 2m(m+1)k^{-3}\left(\frac{1}{(1+k^{-1})(1+2k^{-1})}\right)$$

This implies that p_k is proportional to k^{-3} with a correction factor. Note however that as $k \to \infty$ the correction term approaches 1. Meaning that for large N the model will predict a value of $\gamma < 3$ such that $|\hat{\gamma} - \gamma|$ is arbitrarily small.

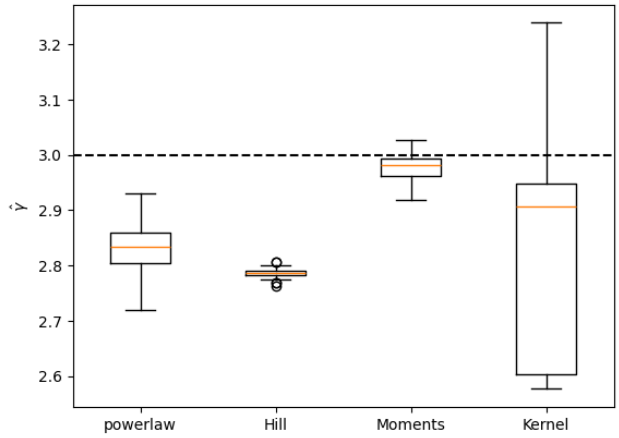


FIG. 2. Boxplot showing the mean and deviation for each of the respective methods. The dashed horizontal line represents the $\gamma = 3$ true value.

TABLE II. Model Estimates and Comparison Results

Model	**Mean**	**Std**	**Failed**
powerlaw	2.832	0.043	0
Hill	2.787	0.007	0
Moments	2.978	0.022	0
Kernel	2.798	0.189	0
Model Comparison Summary			
favor power law ($R > 0$, $p < 0.05$)			0
favor lognormal ($R < 0$, $p < 0.05$)			4
inconclusive ($p \geq 0.05$)			96

(c) i. Which Estimators Recover $\gamma = 3$?:
Moments is the only one to recover $\gamma = 3$. Its mean is 2.98, and its box is on the dashed line/ With 100 networks, the standard error is 0.0022, so the gap is detectable but not as significant as the others.
Powerlaw and Hill are both biased towards a lower estimate, with their respective mean being 2.83 and 2.79. Both of their boxes are entirely below 3, and their standard errors are 0.17 and 0.21, respectively. Note that Hill is also very precise, with a standard deviation of 0.007, so it is consistently wrong.
Kernel has a mean of $2.80 < 3$ but it has a huge spread (std = 0.19). The problem for Kernel Type Estimator is mostly with its variance and not its bias.

ii. Why? We start by taking the natural logarithm of p_k:

$$ln(p_k) = ln(2m(m+1)) - ln(k) - ln(k+1) - ln(k+2)$$

To determine the slope with respect to k, we find $\frac{\partial ln(p_k)}{\partial ln(k)}$

$$-(1 + \frac{k}{k+1} + \frac{k}{k+2}) = -(1 + \frac{1}{1+\frac{1}{k}} + \frac{1}{1+\frac{2}{k}})$$

Note that as $k \to \infty$, this slope will approach -3 from below. Hill and Powerlaw both assume a pure power law above the cutoff k_{min}, so they average these local slopes and land below 3. Moments is built with deviation from a pure power law in mind, which explains why it is less affected

iii. What does the likelihood ratio test conclude?
Out of 100 networks: 0 significantly favor a power law, 4 significantly favor a lognormal, and 96 are inconclusive (p$\geq$ 0.05). The test never confirms a power law. Most of the time, is cant distinguish a power law from a lognormal. The reason is that the tail above the k_{min} contains only a few nodes, and the exact p_k curves away from the straight line that a 2-parameter lognormal model can better mimic. Conclusion: At finite sizes, the test has difficulty detecting a true power law.

(d) BnC treats scale-free networks as one whose degree distribution follows $p_k \propto k^{-\gamma}$ in its tail. They make this operational as a graded hierarchy running from "super weak" to "strongest". The strong versions require a statistically plausible power-law fit (tail $>$ 50 nodes, $\gamma \in [2, 3]$, no competing distribution). To test their definition, they used the Clauset- Schalzi-Newmman procedure.
Vol defines scale-free via extreme value theory. The tail is regularly varying : $P(K > k) = l(k) \cdot k^{-(\gamma-1)}$, with $l(k)$ being a slowly varying function. A pure power law is a special case, but any distribution whose tail behaves like $k^{-\gamma}$ asymptotically up to slowly varying corrections count. They estimate the tail index with Hill, moments, and kernel-type estimators.
They disagree because they are working from two different definitions (something I'm beginning to think is common in this field), BnC asks whether the data are well described by an exact power law above k_{min} and better than the alternatives. Vol asks whether the tail is asymptotically power-law like.
Our networks distribution is asymptotically k^{-3} (with correction factor 1-3/k). So it is regularly varying and scale-free according to Vol's definition. But we don't meet the criteria for BnC's definitions, as none of our tests preferred a power law over the lognormal model.
I understand the need for both definitions in the real world; something that follows a true power law is very rare due to the structural limitations of the real world. However, data that is power-law-like exhibits the behavior of interest (hubs, diverging second moments, robustness, etc.) One could reasonably call this data scale-free even though it is not described by a true power law. For my own sanity, I will say that scale-free networks are well done, but true power law networks are rare.

2. *Question 2.*

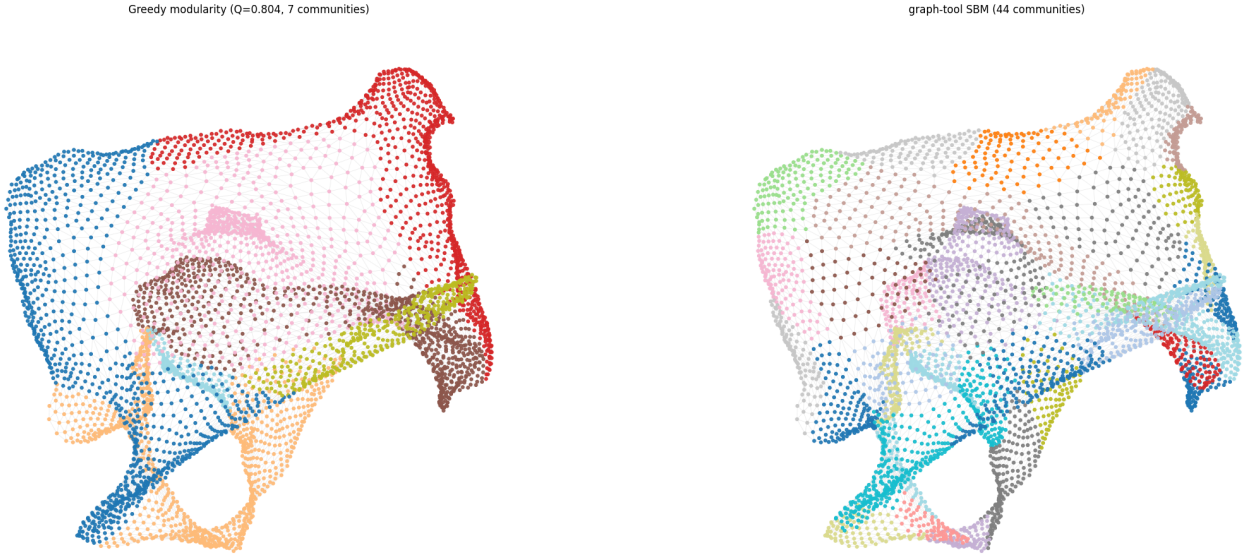


FIG. 3. On the left hand side, is a visualization of the 3elt graph with greedy modularization. A total of 7 communities were detected with a modularity score : 0.8041. On the right hand side is a visualization of the 3elt graph with graph tools SBM community partition. A total of 44 communities were detected with a modularity score of 0.8875.

(a)

TABLE III. Comparison of Community Detection Methods

Label	**Method**	**K**	**Q**
(a)	greedy, orig	7	0.8041
(b)	graph-tool, orig	44	0.8875
(c)	greedy, rand	16	0.3930
(d)	graph-tool, rand	1	0.0000

(b)

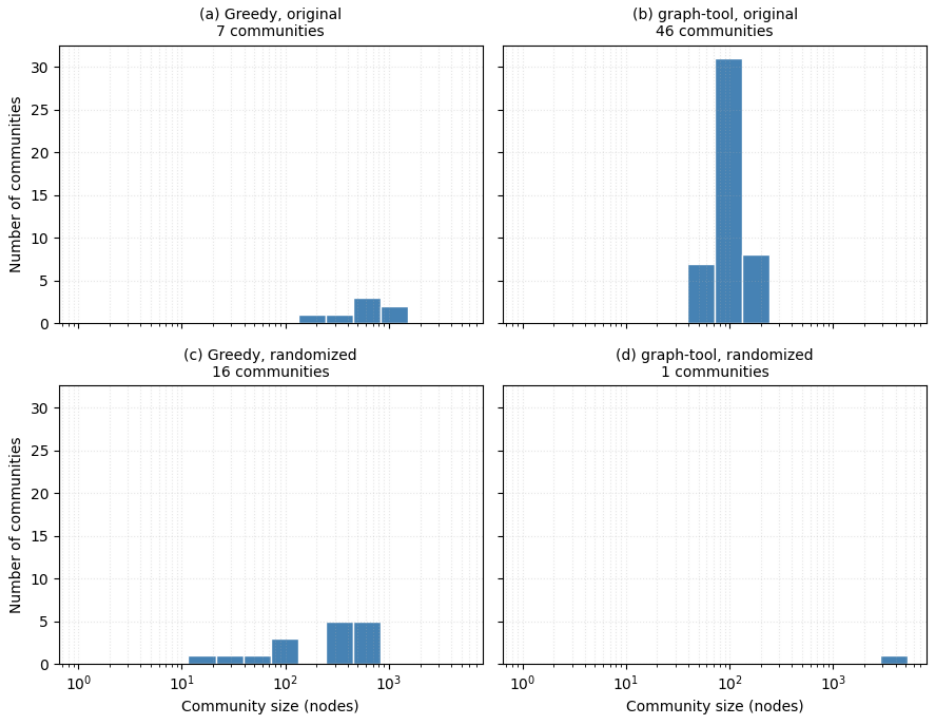


FIG. 4. The four plots above represents the size of a community vs the number of communities in respective graph partitions. The x-axis has a logarithmic scale while the y-axis has a linear scale.

(c) 3elt is a finite element mesh, from the graph partitioning benchmark archives. It has about 4.7K nodes, max degree 9, and nearly uniform degrees. This gives it two properties that produce strong communities:

Geometry creates small cuts. You can slice a 2D mesh into patches, and the number of edges crossing a patch boundary grows like the perimeter (about $\sqrt{n}$ for n nodes), while the edges inside grow like the area (n). So patches are much more internally connected than externally, which is exactly what modularity rewards. This is why Q is so high (0.80 to 0.89). There are no hubs and no hierarchy. Every region looks like every other region, so the "natural" communities are roughly equal-sized patches.

(a) greedy, original: 7 large communities, 200 to 1,500 nodes, broad spread. The CNM greedy algorithm repeatedly merges the pair of communities that most increases Q, and it tends to grow a few big communities. Modularity also has a resolution limit, so it favors merging small patches into larger ones. And since the mesh has no sharply separated clusters, many partitions have nearly the same Q, so the greedy path lands on an uneven one. This is also why it scored lower than the SBM (0.804 vs 0.888) despite optimizing modularity directly.

(b) graph-tool, original: about 44 communities, tightly clustered around 100 nodes. The SBM fits a statistical model and picks the number of blocks that

best compresses the graph. In a homogeneous mesh, the best fit is many similar patches, so the sizes concentrate in a narrow band. (Note: The SBM inference is stochastic, so a rerun can give a slightly different K.)

(c) greedy, randomized: 16 communities, sizes from about 10 to 800. Degree-preserving randomization keeps each node's degree but rewires who it connects to, which destroys the geometry. The graph becomes random-like and has no small cuts. Greedy modularity still returns Q = 0.39 because modularity maximization finds partitions that look good even in random graphs (random fluctuations in edge density are enough). The broad, irregular size distribution reflects that these communities are artifacts rather than real structure, so nothing pushes them toward a natural size.

(d) graph-tool, randomized: 1 block of about 4,700 nodes. The SBM's description-length criterion penalizes model complexity. On the randomized graph, splitting into blocks doesn't compress the graph any better than a single block, so it reports no community structure. That's the single bar at the far right of the plot.

The comparison across the four plots shows the structure in 3elt comes from its geometry, since randomizing removes it. It also shows that greedy modularity can report communities where none exist (randomized model), or under report the actual number of communities (the original model). While the SBM does not.

(d) To compare partitions quantitatively, I would use the adjusted Rand index (ARI) ([1]), a standard measure in the clustering and community detection literature(and easy to understand). ARI considers every pair of nodes and checks whether the two partitions agree about that pair (both place the nodes in the same community, or both place them in different communities). It then corrects this agreement for what would be expected by chance.

A function $f(b_1, b_2)$ for comparing partitions should have these properties:

Symmetry: $f(b_1, b_2) = f(b_2, b_1)$, since neither partition is the reference.
Label invariance: renaming community IDs must not change the score. "Community 3" in one partition has no relation to "community 3" in another, so f must depend only on which nodes are grouped together.
Identity: f gives its maximum similarity only when the partitions are identical.
Works with different numbers of communities: our partitions have K = 7, 44, 16, and 1, so f can't assume equal K.
Chance correction: two unrelated partitions should score near zero, regardless of how many communities they have. Without this, partitions with many small communities appear to agree more than they should.
Bounded and interpretable: a fixed scale (ARI is at most 1) lets us compare values across pairs and networks.

I would compute ARI for all pairs of the four partitions. (a) vs (b) shows whether

the two methods agree on the real network. Original vs randomized for the same method shows whether the method finds structure beyond the degree sequence, and a low ARI is expected. Comparing (c) with (d) is uninformative because (d) is a single block.

3. *Question 3.*

(a) **Motivation:** Real networks rarely come with known ground-truth communities. People therefore test on synthetic graphs with planted communities. The standard one, the Girvan-Newman (GN) benchmark, is unrealistic in three ways:

- All nodes have roughly the same degree (a Poisson-like distribution).
- All communities have the same size (4 groups of 32).
- The graph is tiny (128 nodes).

GN is also too easy: many algorithms score about the same on it, so it can't discriminate between them.
The main contributions of the paper are :

- A benchmark with power-law degree distribution (exponent γ) and power-law community-size distribution (exponent β), with a tunable mixing parameter μ.
- A generation procedure that scales linearly with the number of edges, so graphs with 10^5 to 10^6 nodes are feasible.
- A demonstration that the benchmark is much harder than GN. Modularity optimization and Potts-model clustering both fail at small μ, and they get worse as N grows and as communities become more similar in size. Modularity's resolution limit shows up clearly, because small communities get merged.

Inputs:

- N, the number of nodes
- $\langle k \rangle$, the average degree
- γ, the degree exponent (paper: $2 \leq \gamma \leq 3$)
- β, the community-size exponent (paper: $1 \leq \beta \leq 2$)
- μ, the mixing parameter
- $k_{\min}$, $k_{\max}$, $s_{\min}$, $s_{\max}$, which the paper chooses to satisfy the constraints on $\langle k \rangle$ and N (in implementations they are often exposed as `min_degree`, `max_degree`, `min_community`, `max_community`)

The result is a graph on N nodes, plus a ground-truth partition into communities. Each node has an internal-degree share of about $1 - \mu$ within its community. An algorithm is scored by comparing its recovered partition to the planted one, with normalized mutual information.

(b) Sampling a power law is not enough, because the sizes must be consistent with the degree sequence and with N. Sizes must sum to exactly N, so that every

node can be placed. In practice, you resample or adjust the sizes until they do. $s_{min} > k_{min}$ and $s_{max} > k_{max}$. A node with internal degree k_{in} needs at least k_{in} other members in its community. If a community is no larger than the node's degree, that node cannot fit, with its internal links, in any community. This constraint guarantees every node of every degree has at least one community it can join. s_{min} and s_{max} therefore cannot be chosen independently of the degree distribution. Even then, nodes are assigned by the homeless-node procedure of Step 4, which kicks nodes out of full communities, so assignment can take many iterations.

(c) According to the original LFR paper, the generator relies on a configuration-model-like stub-matching procedure constrained by strict overlapping power-laws. It struggles because:

1. The Community-Size vs. Internal Degree Constraint: For each node i, its internal degree $k_{in}^{(i)} = d_i(1-\mu)$ must be strictly less than the size of the community C assigned to it (specifically, $k_{in}^{(i)} < |C|$ for simple graphs without self-loops or multi-edges). If we draw a high node degree from the power law (τ_1) but assign it to a small community drawn from the community size distribution (τ_2), the constraint is violated, and the generator restarts or fails.

2. Strict Stub Matching: LFR attempts to wire internal and external stubs separately to maintain exact community identities and mixing rates. In small networks or networks with highly restricted distributions, the pool of available stubs cannot match perfectly without creating multi-edges or self-loops (which are discarded). Thus, the generator repeatedly rewires and eventually exceeds the 'max_iters' threshold.

The specific parameter choices / combinations that cause LFR to struggle are small N or small community size. Smaller graphs offer less structural flexibility causing the distinction between community and degree to collapse. Furthermore, a small minimum community size means that nodes with high degree cannot fit into them with out violating the internal / external edge split specified by μ. Even furthermore, if a graph has high average degree but a relatively low graph size or small community structure, then the aforementioned problems will still persist. (I would like to put a table showing grid search results here but i fear the code will never finish running ...)

4. *Question 4.*

(a) here is my package : _malware. (ˆω ˆ)

[1] Hubert, Lawrence. "Comparing Partitions." *Journal of Classification*, Springer Nature, 1985, doi:10.1007/BF01908075.